

Тестовое задание

Инженер по ИИ-агентам · 1 день · Claude Code

Mini-ассистент по образу «Коли» · стек Java 8 / Maven · 28.06.2026

Цель: построить **mini-ассистента по образу «Коли»** — он читает входящую почту через Outlook, обрабатывает запрос через LLM с инструментами (tool-calling) и **отвечает письмом**. Мы оцениваем две вещи независимо: **(1)** умение делать агентов на стеке и по подходам компании и **(2)** умение работать с ИИ (Claude Code).

1. Что вам выдают

- **Windows-среда** с установленным **Outlook** и **тестовым почтовым ящиком** (логин/пароль — отдельным сообщением). На этой среде проверяется живой JACOB на защите.
- Опционально — **LLM-ключ/эндпоинт**. Можно использовать свой ключ **или мок** (см. §5.2).
- Этот документ + примеры входящих писем (§10).
- Предустановленная нативная библиотека **JACOB** (`jacob-1.20.x64.dll` на PATH) — чтобы вы не тратили день на её сборку.

Стартуете с **пустого репозитория**.

2. Обязательный стек (заданные технологии)

- **Java 8** (именно 8), **Maven**, итоговый **fat-jar** (maven-shade).
- LLM по **HTTP** (okhttp или эквивалент).
- Конфиг в **YAML** (jackson/snakeyaml).
- Outlook через **COM/JACOB** (`net.sf.jacob-project:jacob:1.20`).
- Тесты — **JUnit 4**. Логи — **slf4j/logback**.

3. Что строит ассистент (ЯДРО — обязательно)

Цикл: **опросить входящие** → на каждое **Непрочитанное письмо** прогнать **LLM tool-loop** с телом письма как запросом → **ответить письмом отправителю**. Письмо = единица диалога.

3.1 Канал (основной) — Outlook COM/JACOB

- За интерфейсом `MailChannel`: как минимум `List<Msg> fetchUnread()` и `void reply(Msg, String body)`.
- Реализации: `OutlookMailChannel` (JACOB) и `MockMailChannel` (для тестов).
- **Идемпотентность (обязательно)**: каждое письмо обрабатывается **ровно один раз**; состояние «обработанных» переживает рестарт процесса (seen-store на диске). Ключ дедупликации — **стабильный идентификатор письма** (Outlook `EntryID` / `Message-ID`), не тема/тело. Без этого poll-loop будет отвечать на одно и то же письмо каждый цикл — это считается дефектом ядра.

3.2 LLM

- Интерфейс `LlmClient`: `chat(messages, tools) → response`. Реализации: HTTP и мок.
- Ключ/секрет — **только из переменной окружения** (имя env задаётся в конфиге).

3.3 Инструменты (≥2; контракты можно спроектировать самостоятельно)

Рекомендуемый набор, детерминированный и тестируемый: - `current_datetime()` → текущее время ISO (через **инжектируемый Clock** — чтобы тесты были детерминированы); - `add_reminder(text, dueIso)` → запись в локальный JSON-стор; - `find_items(query)` → поиск по стору.

3.4 Tool-loop

- Ограничение числа шагов `maxSteps`.
- **Устойчивость к кривому/галлюцинированному tool_call**: не падать — вернуть модели ошибку и корректно завершить.

3.5 Конфиг (YAML)

`llm.endpoint/model/apiKeyEnv/timeoutMs, agent.maxSteps, store.path, mail.pollSeconds/profile/folder`. **Никаких секретов в коде или git.**

3.6 Graceful-фолбэк

- LLM недоступен/таймаут → понятный ответ-письмо **или** пропуск + WARN; **никаких stacktrace** пользователю.
- Ошибка COM → WARN, цикл **не падает** (обрабатываем следующее письмо).

3.7 ПДн и безопасность (обязательно)

- Письма содержат **персональные данные**. **Не логировать тела писем/ПДн в открытом виде** (не писать в лог, либо маскировать).

- Секреты — только из env.
- **Аудит:** append-only журнал действий (какое письмо обработано, какие `tool_call`). Желательно с **цепочкой хешей** (hash-chain) для неизменяемости — по образцу `HmacSigner` в «Коле».

3.8 Логи

- Структурные **event-keys** (напр. `agent_mail_seen`, `agent_tool_call tool=...`, `llm_failed ...`). Без ПДн.

4. Обязательные подходы (оцениваются ОТДЕЛЬНО, см. рубрику)

- **Plan-first:** короткий план/issue-лист **до кода** в `PLAN.md`. (**beads не требуется.**)
- **TDD:** тест **перед** реализацией; переход **red→green** **виден в git-истории**. Нет прод-кода без предварительно падающего теста.
- **Сверка с документацией** (Context7 / официальная дока) перед использованием библиотеки.
- **Graceful-фолбэки, структурные логи, секреты из env.**
- **security-review перед сдачей:** само-ревью на секреты, ПДн в логах, инъекции в tool-аргументах.
- **verification-before-completion:** запускайте команды и показывайте вывод; не заявляйте «работает», не предъявив доказательства.
- **Вменяемые атомарные коммиты** (не один гигантский «AI-dump»).

5. Тесты (обязательно; ЗЕЛЁНЫЕ без Outlook)

`mvn test` должен проходить на машине **без Outlook** (как CI). Покрыть минимум: - юниты инструментов; - tool-loop на **мок-LLM** (скрипт: `tool_call` → финальный ответ); - канал на `MockMailChannel` (письмо-вход → ожидаемый ответ); - загрузку конфига; путь фолбэка; - **идемпотентность** (одно письмо не обрабатывается дважды).

Подсказка по среде: JACOB — нативный COM-мост, на Linux его статический инициализатор зовёт `System.exit`. Исключите артефакт `jacob` из тест-classpath (как сделано в «Коле» через `classpathDependencyExcludes` в surefire), чтобы тесты шли в любой среде.

6. Что ВНЕ scope (не делать — экономьте день)

Реальный Telegram, Confluence, календарь, DPAPI/cookies, RAG/эмбединги, БД сложнее JSON-файла, мультипользовательность, OAuth/SSO, веб-панель, деплой сверх fat-jar. Один инстанс, один ящик.

7. Stretch (опционально — сигнал на senior)

retry/timeout/backoff для LLM; allow/deny-gate инструментов из конфига; память диалога по отправителю/треду; OpenTelemetry-span; override конфига через env-переменные; устойчивость к COM-сбою (reconnect); расширенный аудит.

8. Что сдать

- **Git-репо** с полной историей (атомарные коммиты, видимый red→green).
- **PLAN.md** (план до кода).
- **Экспорт сессии Claude Code** (промпты/планы) — например `/export` или файлы `~/.claude/projects/<проект>/*.jsonl`.
- **README.md**: `build / run / test` + раздел «**Как я работал с ИИ**» (стратегия промптов, что проверяли у модели, что отклонили).

9. Защита (15-20 минут)

- Разбор архитектуры и границ.
- Объяснение **одного теста** (зачем он, что ловит).
- **Живой JACOB-прогон**: реальное письмо в ящик → ассистент отвечает.
- **Живое расширение**: «добавьте инструмент `delete_reminder` с помощью Claude Code прямо сейчас».

10. Примеры входящих писем (golden)

1. «Напомни завтра в 10 позвонить Ивану» → `add_reminder`, ответ-подтверждение с датой.
2. «Что у меня запланировано?» → поиск/список, ответ списком.
3. «Какое сегодня число?» → `current_datetime`.
4. Пустое/мусорное письмо → graceful, без падения, без лишних действий.

11. Чек-лист готовности (само-проверка перед сдачей)

- ☐ `mvn package` → fat-jar, запускается
- ☐ `mvn test` зелёный **без Outlook**
- ☐ `MailChannel`: JACOB-реализация + мок
- ☐ ≥2 инструмента, tool-loop работает на моке
- ☐ идемпотентность (seen) + переживает рестарт
- ☐ конфиг-driven, секреты из env, в git ничего секретного
- ☐ graceful-фолбэк на LLM **и** COM
- ☐ структурные логи, **без ПДн**

- ☐ аудит-журнал действий
- ☐ `PLAN.md` + экспорт сессии Claude Code + `README`